

Memoria

Sistema Domótico IoT con Integración en Discord

Daniel Aquino Santiago y Enrique Gómez Fernández

ÍNDICE

ÍNDICE.....	2
1. Introducción.....	3
2. Actores.....	3
3. Requisitos Funcionales.....	3
4. Decisiones de diseño.....	4
5. Casos de uso.....	6
1. Switch.....	6
2. Sensor.....	6
3. Clock.....	6
6. Esquema de Datos de Persistencia.....	7
1. Dispositivos (devices.json).....	7
2. Reglas (rules.json).....	7
3. Eventos (events.json).....	7
7. Clases Principales y Relación entre Ellas.....	7
8. Implementación.....	8
1. Arquitectura.....	8
2. Componentes Clave.....	9
3. Flujo de Datos.....	9
9. Limitaciones.....	9
10. Conclusiones.....	9
Conclusiones personales.....	9
Conclusiones Técnicas.....	10

1. Introducción

Este documento recoge el análisis de requisitos, decisiones de diseño e implementación para un sistema domótico basado en IoT que utiliza MQTT como protocolo de comunicación y ofrece interacción a través de un bot en Discord. Se detallan los actores, casos de uso, requisitos funcionales y no funcionales, y las restricciones específicas de la opción Discord.

2. Actores

- **Dispositivo IoT (Sensor/Interruptor/Reloj)**: Publica cambios de estado y recibe comandos a través de MQTT.
- **Controller (Gestor central)**: Suscriptor y publicador MQTT. Persiste datos, genera eventos y comunica con el Rule Engine.
- **Rule Engine**: Comprueba reglas "dado-cuando-entonces" y dispara acciones sobre dispositivos.
- **Bridge (Traductor)**: Enlaza el sistema con la API de Discord, traduce eventos y comandos.
- **Bot de Discord**: Pública notificaciones y recibe comandos de usuarios en un canal específico.
- **Usuario Final**: Interactúa escribiendo comandos en Discord y revisando notificaciones.

3. Requisitos Funcionales

1. **RF1 – Gestión de Dispositivos**
 - RF1.1: Registrar, editar y eliminar dispositivos mediante comandos Discord.
 - RF1.2: Almacenar ID, tipo (sensor, interruptor, reloj) y metadatos MQTT.
2. **RF2 – Gestión de Reglas**
 - RF2.1: Crear reglas con operadores ==, >, <.
 - RF2.2: Listar y eliminar reglas.
3. **RF3 – Monitorización y Notificaciones**
 - RF3.1: Recibir eventos MQTT y persistir en base de datos.
 - RF3.2: Enviar notificaciones de eventos y acciones realizadas al canal Discord.
4. **RF4 – Control Remoto**
 - RF4.1: Enviar comandos MQTT desde Discord para cambiar estados de dispositivos.
5. **RF5 – Seguridad y Acceso**

- RF5.1: Sólo usuarios autorizados (roles configurables en Discord) pueden ejecutar comandos críticos.

4. Decisiones de diseño

1. ¿Controller y Rule Engine han de ser aplicaciones separadas?, ¿por qué?, ¿qué ventajas tiene una y otra opción?

- **Opción actual (integrados):** En el código, RuleEngine es una clase instanciada dentro del Controller (línea `self.rule_engine = RuleEngine(...)`). No son aplicaciones separadas, sino componentes acoplados en el mismo proceso.
 - **Ventajas:**
 - La comunicación es directa sin overhead de red.
 - No requiere infraestructura adicional.
 - Ambos comparten el mismo estado (base de datos) sin riesgo de desincronización.
 - **Desventajas:**
 - No se pueden escalar independientemente.
 - Cambios en RuleEngine pueden afectar al Controller.
- **Opción separada:**
 - **Ventajas:**
 - Se pueden desplegar más instancias del RuleEngine si hay muchas reglas.
 - **Desventajas:**
 - Requiere API REST/gRPC, manejo de errores de red, etc. Lo que haría su desarrollo más complejo
 - Se puede producir un posible cuello de botella ya que la comunicación HTTP/RPC puede ser más lenta.

La opción que elegimos es adecuada para un este sistema, ya que es innecesario el aumento de dificultad ya que el controlador debe esperar a que el rule_engine acabe para saber si debe mandar o no un cambio. La separación sólo sería necesaria si las reglas son muy complejas o requieren escalar de forma independiente.

2. ¿Cómo se comunican Controller y Rule Engine en la opción escogida?

En la opción actual, la comunicación es directa mediante llamadas a métodos:

- El Controller llama a `self.rule_engine.evaluate_rules(...)` cuando llega un mensaje MQTT (línea 24 del Controller.py).
- El RuleEngine accede a los datos compartidos a través de la clase Persistence (inyectada en ambos).

3. ¿Tiene sentido que compartan funcionalidad?, ¿qué relación hay entre ellos?

- **Funcionalidad compartida:** Ambos usan Persistence para acceder a dispositivos/reglas. Esto es correcto, ya que centraliza la gestión de datos.
- **Relación:** El Controller es el "dueño" del RuleEngine (composición). El RuleEngine es un servicio especializado que el Controller usa para evaluar reglas. No deberían compartir más funcionalidad para mantener responsabilidades separadas.

4. ¿Cuántas instancias hay de cada componente?

- **Controller:** 1 instancia (es un singleton en Controller.py, creado en controller = Controller(...) y ejecutado en main()).
- **RuleEngine:** 1 instancia (creada dentro del Controller).
- **Bridge:** 1 instancia (Flask + Discord bot en bridge.py).
- **Persistence:** 1 instancia por componente (el Controller y el RuleEngine comparten la misma instancia inyectada).

5. ¿Controller y Bridge han de ser aplicaciones separadas?, ¿por qué?, ¿qué ventajas tiene una y otra opción?

- **Opción actual (separados):**
 - **Ventajas:**
 - **Separación de responsabilidades:** El Bridge maneja Discord y el Controller maneja MQTT.
 - **Resiliencia:** Un fallo en el Bridge no afecta al Controller.
 - **Escalabilidad:** Se pueden desplegar en distintos servidores.
 - **Desventajas:**
 - **Complejidad:** Requieren comunicación HTTP (ej: requests.post("http://localhost:8081/add_device")).
- **Opción integrada:**
 - **Ventajas:**
 - **Menos overhead:** No hay llamadas HTTP internas.
 - **Desventajas:**
 - **Acoplamiento:** Cambios en la API de Discord afectarían al Controller.

El Bridge actúa como un adaptador entre Discord y el Controller, lo que sigue el principio de responsabilidad única. Por ello, deben ser aplicaciones separadas.

6. ¿Cómo se comunican Controller y Bridge en la opción escogida?

Se comunican mediante **HTTP REST**:

- El Bridge envía comandos al Controller mediante POST/GET a http://localhost:8081 (ej: add_device, set_switch, etc.).
- El Controller notifica eventos al Bridge mediante POST a http://localhost:5000/event (línea 28 del Controller.py y ruta /event en bridge.py).

5. Casos de uso

1. Switch

Flujo principal:

1. El usuario envía un comando para encender/apagar el interruptor
2. El sistema envía el comando vía MQTT
3. El dispositivo cambia su estado (ON/OFF)
4. El dispositivo publica su nuevo estado
5. El sistema registra el evento
6. El sistema notifica el cambio al usuario por Discord

2. Sensor

Flujo principal:

1. El sensor mide un valor (temperatura, humedad, etc.)
2. El sensor publica periódicamente su lectura actual
3. El sistema recibe y almacena el valor
4. El sistema evalúa reglas
5. Si se cumple alguna regla, el sistema ejecuta acciones asociadas
6. Genera un evento y lo envía al bridge
7. El sistema notifica el cambio al usuario por Discord

3. Clock

Flujo principal:

1. El reloj actualiza y publica la hora actual
2. El sistema recibe la hora
3. El sistema evalúa reglas basadas en horarios
4. Si se cumple alguna regla temporal, el sistema ejecuta acciones asociadas
5. Genera un evento y lo envía al bridge
6. El sistema notifica el cambio al usuario por Discord

6. Esquema de Datos de Persistencia

El sistema utiliza tres archivos JSON para almacenar su estado:

1. Dispositivos (devices.json)

```
[
  {
    "id": "redes2/grupo/pareja/123",
    "type": "switch|sensor|clock",
    "state": "ON|OFF|valor numérico|hora"
  }
]
```

2. Reglas (rules.json)

```
[
  {
    "condition": "if sensor 1 > 25",
    "action": "switch 2 ON"
  }
]
```

3. Eventos (events.json)

```
[
  {
    "device_id": "redes2/grupo/pareja/123",
    "state": "ON|OFF|valor",
    "timestamp": "1620000000.0"
  }
]
```

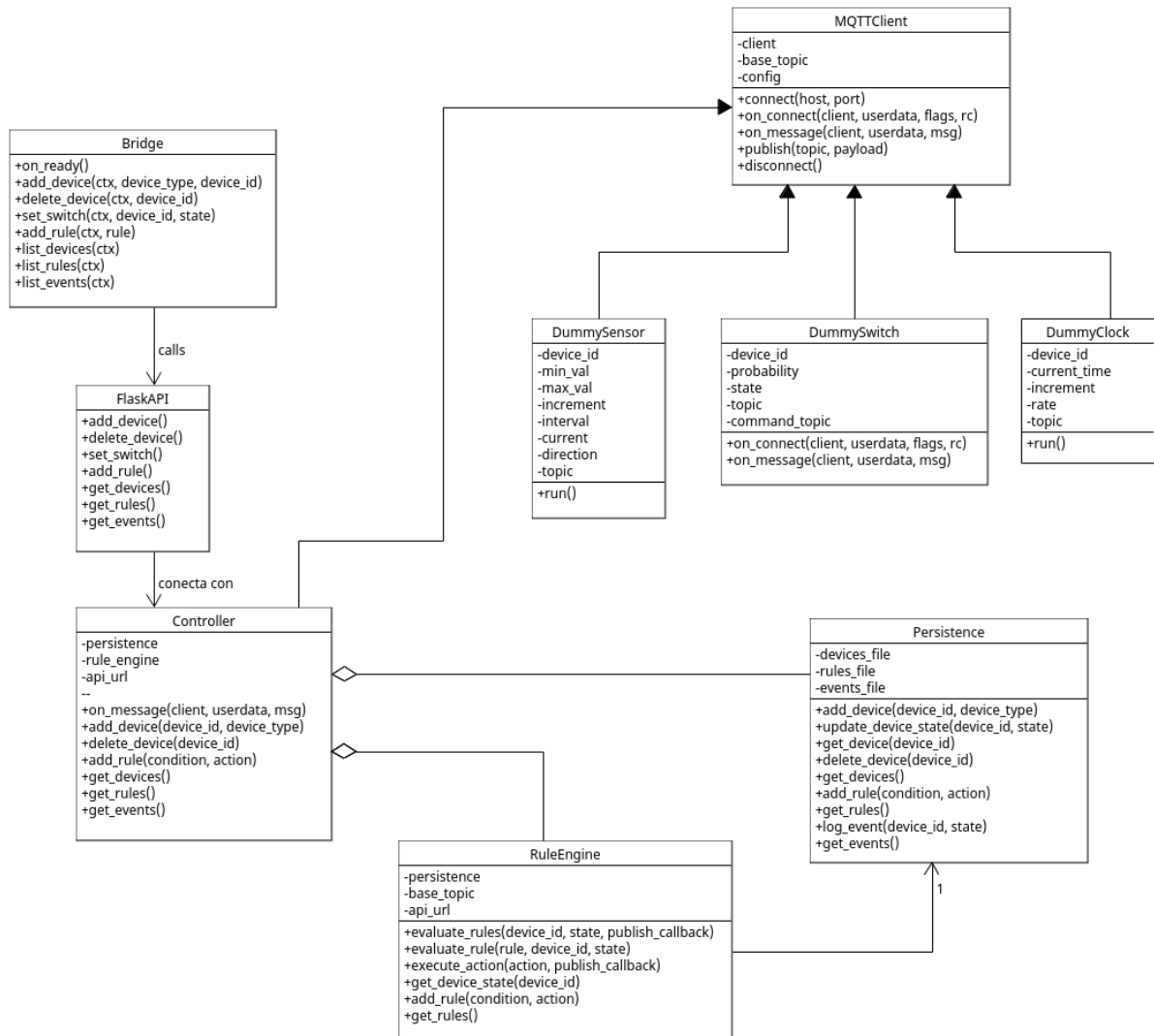
7. Clases Principales y Relación entre Ellas

Relaciones principales:

- **MQTTClient** es la clase base que proporciona la funcionalidad MQTT
- **Controller** hereda de MQTTClient y coordina el sistema
- **DummySwitch**, **DummySensor**, y **DummyClock** heredan de MQTTClient para implementar dispositivos específicos

- **Controller** utiliza **Persistence** para almacenar el estado
- **Controller** utiliza **RuleEngine** para evaluar reglas
- **RuleEngine** utiliza **Persistence** para acceder a las reglas

Lo veremos mejor en el siguiente diagrama de clases:



8. Implementación

1. Arquitectura

El sistema implementa una arquitectura de microservicios con:

- Servicios independientes (dispositivos, controlador, bridge de Discord)
- Comunicación basada en mensajes MQTT
- APIs REST para integración externa

2. Componentes Clave

- **MQTT:** Protocolo de comunicación central para dispositivos IoT
- **Flask:** Servidor web para APIs REST
- **Discord Bot:** Interfaz de usuario para comandos y notificaciones desarrollado mediante el bridge
- **Rule Engine:** Motor de reglas para automatización
- **Controller:** Maneja MQTT, reglas, la API Flask y la persistencia de los dispositivos

3. Flujo de Datos

1. Los dispositivos publican su estado actual a través de MQTT
2. El controlador recibe estos cambios y actualiza la persistencia
3. El controlador evalúa las reglas para ejecutar acciones
4. Las acciones se traducen en comandos MQTT a los dispositivos
5. Los eventos se notifican a través de Discord

9. Limitaciones

- Se usan archivos JSON, lo que limita la escalabilidad para grandes cantidades de dispositivos o reglas.
- Solo soporta comparaciones simples ($>$, $<$, $=$) y acciones de conmutación. No hay soporte para condiciones complejas o múltiples acciones.
- No hay implementación de autenticación o cifrado en la comunicación MQTT o en las APIs REST.
- No hay reconciliación de estado cuando hay desconexiones o reinicio del sistema.
- El diseño no contempla múltiples instancias del controlador trabajando en paralelo.
- Un dispositivo en sí no tiene persistencia, al arrancar elige un estado y lo comunica

10. Conclusiones

Conclusiones personales

Como equipo, hemos logrado alcanzar los objetivos de aprendizaje propuestos mediante la implementación de este sistema IoT. El trabajo con MQTT nos ha permitido comprender a fondo este protocolo esencial en entornos IoT, experimentando directamente sus ventajas para la comunicación asíncrona entre componentes. La división de responsabilidades para gestionar los diferentes módulos del sistema (dispositivos, controlador, API y bot) nos obligó a establecer interfaces claras y a coordinar eficazmente nuestro trabajo, simulando un entorno profesional. La programación en Python para construir esta arquitectura distribuida consolidó nuestros conocimientos teóricos de la asignatura, poniendo a prueba nuestra capacidad para aplicarlos en un caso práctico complejo. Nuestra metodología de desarrollo,

basada en pruebas continuas con dispositivos virtuales, nos permitió identificar y corregir problemas tempranamente mientras avanzábamos en paralelo en distintos componentes. Finalmente, la implementación del bot de Discord nos introdujo a las APIs abiertas de plataformas populares demostrando que podemos crear interfaces de usuario efectivas integradas con sistemas IoT distribuidos.

Conclusiones Técnicas

Desde la perspectiva técnica, nuestro trabajo en equipo nos ha permitido evaluar las tecnologías implementadas. El protocolo MQTT ha demostrado ser una elección acertada como columna vertebral de comunicación, proporcionando el desacoplamiento necesario entre los distintos componentes y facilitando nuestra división de tareas durante el desarrollo. La combinación de Flask para la API REST y Discord para la interfaz de usuario resultó ser flexible y potente, aunque encontramos desafíos de sincronización que requirieron soluciones creativas utilizando hilos y eventos asíncronos. La persistencia basada en archivos JSON, que inicialmente adoptamos por su simplicidad, reveló ser una limitación importante que comprometería la escalabilidad y robustez del sistema en un entorno de producción real. El motor de reglas que desarrollamos funciona correctamente para casos de uso básicos, pero quedó evidente que implementar condiciones más complejas requeriría un rediseño. A través de extensas pruebas, identificamos vulnerabilidades en el manejo de errores, especialmente en escenarios de desconexión de la red o fallos en los dispositivos.